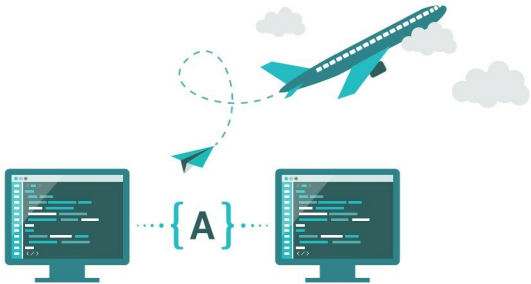


An Introduction to gRPC

Remote Procedure Call (RPC) has been around for decades as a mechanism for inter-process communication in applications. The client making a call talks to a stub that works behind the scenes to handle the marshalling/unmarshalling of the request/response data. To the client it appears that the service is residing locally.



The last decade has seen the rise of public APIs and standards like REST for building HTTP based services. Both XML and JSON are frequently used as the data format for these service methods' request and response data. With the proliferation of these services and the emerging trend towards micro-services, latency becomes an important consideration when creating high performance and efficient services.

To address this, Google had been working for a while to come up with a specification that defines both service interfaces and efficient communication protocols, along with bindings in multiple languages for both the client and server sides. The company later made this available as open source to the general public and the gRPC Project was born. The project helps to implement services in multiple languages with pluggable support for load balancing, health checking and authentication.

The gRPC model is shown in Figure 1.

The service, its methods and the messages are defined via a separate specification known as ProtocolBuffers. This specification is then implemented by the server in one of the

supported languages. Most modern programming languages like Java, Python, C#, Node.js and others are supported. You can either generate the server bindings via the tools provided or even dynamically provide the implementation.

On the client side, you can generate the gRPC stub in a client language of your choice, and use that stub directly in your client to invoke the service.

Why use gRPC?

gRPC has several advantages that are worth considering in your application. Some of them are:

- It is built on HTTP/2, which provides us with a high speed communication protocol that can take advantage of bi-directional streaming, multiplexing and more.
- The latest version of ProtocolBuffer, i.e., version 3, supports many more languages.
- The ProtocolBuffer data has a binary format and, hence, provides a much smaller footprint than JSON/XML payloads that are currently the most popular. This can make a big difference when latency is an issue in your API.